Relatório de Segurança de Dados

Projeto	Vehicle Resale API
Disciplina	Fase 5 – Dados e Segurança da Informação
Curso	Pós-Graduação FIAP – Arquitetura de Software
Data	Março/2026

Sumário

1. Dados Armazenados pela Solução
2. Dados Sensíveis e Classificação
3. Políticas de Acesso Implementadas
4. Políticas de Segurança Operacional
5. Riscos Identificados e Mitigações
6. Conformidade com a LGPD
7. Conclusão

---

1. Dados Armazenados pela Solução

A solução utiliza **dois bancos de dados PostgreSQL independentes**: um para dados de identidade (Keycloak) e outro para dados transacionais de negócio (API). Essa separação é intencional e reduz a superfície de exposição de cada banco.

1.1 Banco transacional da API — `vehicle_resale_db`

Entidade `vehicles` — dados de estoque

Campo	Tipo	Contém dado sensível?
id	Long	Não
brand, model, color	String	Não
year	Integer	Não
price	BigDecimal	Não
status	AVAILABLE / SOLD	Não
createdAt, updatedAt, deletedAt	LocalDateTime	Não

Entidade `customers` — dados do comprador

Campo	Tipo	Dado sensível	Base legal (LGPD)
id, userId	Long / String	Não	—
name	String	**Sim**	Art. 5º, I — dado pessoal
email	String	**Sim**	Art. 5º, I — dado pessoal
cpf	String	**Sim**	Art. 5º, I — dado pessoal
phone	String	**Sim**	Art. 5º, I — dado pessoal
address, city, state, zipCode	String	**Sim**	Art. 5º, I — dado pessoal
active	Boolean	Não	—
createdAt, updatedAt	LocalDateTime	Não	—

Entidade `sales` — dados da transação de venda

Campo	Tipo	Dado sensível	Observação
id, vehicle_id	Long	Não	—
buyerName	String	**Sim**	Replicado de `Customer.name`
buyerEmail	String	**Sim**	Replicado de `Customer.email`
buyerCpf	String	**Sim**	Replicado de `Customer.cpf`
saleDate, salePrice	Date / Decimal	Não	—
paymentCode	UUID	Parcial	Confidencialidade operacional
paymentStatus	PENDING/APPROVED/REJECTED	Não	—
createdAt, updatedAt	LocalDateTime	Não	—

1.2 Banco de identidade — `keycloak_db`

Gerenciado exclusivamente pelo **Keycloak 23**. Armazena: usuários, credenciais (hash bcrypt), roles (`admin`, `buyer`) e tokens de sessão.

Importante: o Keycloak **não armazena** CPF, endereço nem dados financeiros da aplicação. Qualquer comprometimento do banco de identidade não expõe dados pessoais de negócio.

---

2. Dados Sensíveis e Classificação

2.1 Mapa de dados sensíveis

Dado	Armazenado em	Finalidade
CPF	`Customer.cpf` e `Sale.buyerCpf`	Identificação e emissão de documentação na retirada
Nome completo	`Customer.name` e `Sale.buyerName`	Identificação e documentação
E-mail	`Customer.email` e `Sale.buyerEmail`	Contato e comunicação
Telefone	`Customer.phone`	Contato
Endereço completo	`Customer.address/city/state/zipCode`	Documentação e entrega
paymentCode (UUID)	`Sale.paymentCode`	Rastreamento da transação

Todos os campos marcados como dado pessoal estão sujeitos à LGPD (Lei nº 13.709/2018, Art. 5º, I).

2.2 Recomendações para produção

Mascaramento em logs: CPF e e-mail não devem aparecer em texto plano nos logs de aplicação. Recomenda-se uso de filtros de log ou mascaramento do tipo `***.***.***.***-**`.

Criptografia em repouso: habilitar TDE (Transparent Data Encryption) no PostgreSQL em produção — disponível nativamente no Amazon RDS com AWS KMS, sem alteração no código da aplicação.

Criptografia em nível de aplicação (opcional): para CPF, pode-se aplicar AES-256 antes de persistir, armazenando apenas o valor cifrado no banco. Exige gerenciamento de chaves via AWS Secrets Manager ou KMS.

Retenção e exclusão: definir política de retenção conforme Art. 16 da LGPD. Implementar endpoint de exclusão ou anonimização de `Customer` quando solicitado pelo titular dos dados.

---

3. Políticas de Acesso Implementadas

3.1 Modelo de autenticação e autorização

O acesso à API é controlado em duas camadas:

1. **Autenticação** — realizada pelo **Keycloak 23** via protocolo OAuth2/OIDC. O cliente obtém um token JWT que contém as roles do usuário.
2. **Autorização** — aplicada pela própria API Quarkus. Cada requisição tem o token validado e a permissão verificada por path e método HTTP, conforme configuração em `application.properties`.

As roles disponíveis são: `admin` (administrador do sistema) e `buyer` (comprador cadastrado).

3.2 Matriz de controle de acesso por endpoint

Endpoint	Método	Acesso permitido
`/api/vehicles/available`	GET	Público (sem token)
`/api/vehicles/sold`	GET	Público (sem token)
`/api/vehicles/{id}`	GET	Público (sem token)
`/api/vehicles`	POST	Autenticado — role `admin`
`/api/vehicles/{id}`	PUT, DELETE	Autenticado — role `admin`
`/api/customers`	GET, POST, PUT	Autenticado — `admin` ou `buyer`
`/api/customers/{id}`	GET, PUT	Autenticado — `admin` ou `buyer`
`/api/sales`	POST	Autenticado — `admin` ou `buyer`
`/api/sales/{id}`	GET	Autenticado — `admin` ou `buyer`
`/api/webhook/payment`	POST	Público* — ver nota abaixo
`/health/*` , `/metrics` , `/swagger-ui/*`	GET	Público (sem token)

(*) Atenção — endpoint de webhook: atualmente sem autenticação. Em produção, é obrigatório restringir o acesso por meio de um header secreto compartilhado (`X-Webhook-Secret`), assinatura HMAC ou whitelist de IPs do parceiro de pagamento.

3.3 Configuração de autorização na aplicação

As políticas são definidas em `src/main/resources/application.properties`. Trecho principal:

```
# Caminhos públicos (sem autenticação)
quarkus.http.auth.permission.public.paths=\
  /health/*,/metrics,/openapi,/swagger-ui/*,/q/*
quarkus.http.auth.permission.public.policy=permit

# Gestão de veículos — apenas role admin
quarkus.http.auth.permission.vehicles-admin.paths=\
  /api/vehicles,/api/vehicles/*
quarkus.http.auth.permission.vehicles-admin.methods=POST,PUT,DELETE
quarkus.http.auth.policy.vehicles-admin.roles-allowed=admin

# Clientes e vendas — qualquer usuário autenticado
quarkus.http.auth.permission.customers.paths=\
  /api/customers,/api/customers/*
quarkus.http.auth.permission.customers.policy=authenticated
```

---

4. Políticas de Segurança Operacional

4.1 Separação de responsabilidades

Camada	Responsabilidade	Tecnologia
Identidade	Autenticação, tokens, users, roles	Keycloak 23
Dados transacionais	Veículos, clientes, vendas	Quarkus API + PostgreSQL
Isolamento de dados	Banco da API separado do Keycloak	Dois containers / instâncias RDS

4.2 Proteção contra ataques

Controle de segurança	Implementação
Proteção contra brute force	Keycloak com `bruteForceProtected=true`, `failureFactor` e `maxFailureWaitSeconds` no realm
Expiração de sessão	`accessTokenLifespan` e `ssoSessionIdleTimeout` configurados no realm Keycloak
Validação de entrada	Bean Validation (`@NotBlank`, `@Size`, `@Pattern`) em todos os DTOs
Prevenção de SQL Injection	Hibernate/Panache com queries parametrizadas
Container não-root	Dockerfile executa a aplicação com UID 185
TLS em trânsito	Configurável via `quarkus.oidc.tls.*`; terminação no ALB/Ingress em produção

4.3 Monitoramento e auditoria

Recurso	Detalhe
Health check	`GET /health/ready` e `GET /health/live` via Smallrye Health
Métricas	`GET /metrics` em formato Prometheus
Logs de aplicação	Nível configurável por pacote em `application.properties`
Auditoria de infraestrutura	AWS CloudTrail + GuardDuty (ambiente de produção)

---

5. Riscos Identificados e Mitigações

5.1 Riscos de acesso e autenticação

Risco	Probabilidade	Impacto	Mitigação implementada
Acesso não autorizado à API	Média	Alto	Keycloak + autorização por path e role
Webhook de pagamento falsificado	Alta	Alto	Nenhuma (recomendada: header secreto ou HMAC)
Brute force nas credenciais	Média	Alto	Keycloak com bloqueio automático por tentativas

5.2 Riscos de dados sensíveis

Risco	Probabilidade	Impacto	Mitigação implementada
CPF e e-mail expostos em logs	Alta	Alto	Nenhuma (recomendada: filtro de log / mascaramento)
Dados em repouso sem criptografia	Média	Alto	Nenhuma (recomendada: TDE no RDS via KMS)
Venda com CPF não cadastrado	Baixa	Médio	Validação no `SaleService` — CPF obrigatório

5.3 Riscos de integridade e disponibilidade

Risco	Probabilidade	Impacto	Mitigação implementada
Veículo bloqueado após pagamento falhar	Média	Médio	Compensação SAGA: reverte para AVAILABLE
Acesso direto ao banco de dados	Baixa	Crítico	Banco em VPC privada (recomendada: Security Groups)
Perda de dados / indisponibilidade	Baixa	Alto	Nenhuma (recomendada: RDS Multi-AZ + backup automatizado)

---

6. Conformidade com a LGPD

A Lei Geral de Proteção de Dados (Lei nº 13.709/2018) estabelece obrigações para o tratamento de dados pessoais. A tabela abaixo mapeia os principais princípios ao estado atual da solução.

Princípio (Art. 6º)	Estado na solução	Situação
Finalidade	CPF e endereço coletados para pagamento e retirada	Atendido
Adequação	Dados compatíveis com a finalidade informada	Atendido
Necessidade	Apenas dados estritamente necessários armazenados	Atendido
Acesso do titular	`GET /api/customers/{id}` disponível (autenticado)	Atendido
Qualidade dos dados	`PUT /api/customers/{id}` permite atualização	Atendido
Segurança	Auth Keycloak + autorização + TLS configurável	Parcial
Prevenção	Validação de entrada + brute force no Keycloak	Parcial
Responsabilização	Logs auditáveis; CloudTrail em produção	Parcial
Não discriminação	Não aplicável ao domínio	—

Ações prioritárias para conformidade plena:

1. Autenticar o endpoint de webhook de pagamento.
2. Habilitar criptografia em repouso no banco de produção (RDS + KMS).
3. Implementar endpoint de exclusão ou anonimização dos dados do titular.
4. Documentar o ciclo de vida e a política de retenção dos dados pessoais.

---

7. Conclusão

A solução **Vehicle Resale API** implementa os controles fundamentais de segurança de dados:

- Autenticação robusta via **Keycloak 23** (OAuth2/OIDC) com proteção contra brute force.
- Autorização granular por path e role (`admin` / `buyer`) configurada no framework Quarkus.
- Separação entre identidade (Keycloak) e dados de negócio (API) em bancos e serviços independentes.
- Acesso a dados sensíveis (CPF, e-mail, endereço) restrito a usuários autenticados.

- Resiliência transacional via SAGA de compensação, evitando inconsistências no estoque de veículos.

Status dos controles de segurança:

Controle	Situação
Autenticação — Keycloak / OAuth2 / JWT	Implementado
Autorização por role e path	Implementado
Separação identidade vs. dados de negócio	Implementado
Validação de entrada — Bean Validation	Implementado
Compensação SAGA	Implementado
Criptografia em trânsito — TLS	Configurável (ALB/Ingress)
Criptografia em repouso	Recomendada — RDS KMS
Autenticação do webhook	Pendente
Mascaramento de dados sensíveis em logs	Recomendado

Documento elaborado para a Fase 5 – Dados e Segurança da Informação – Pós-Graduação FIAP – Arquitetura de Software – Março/2026.